

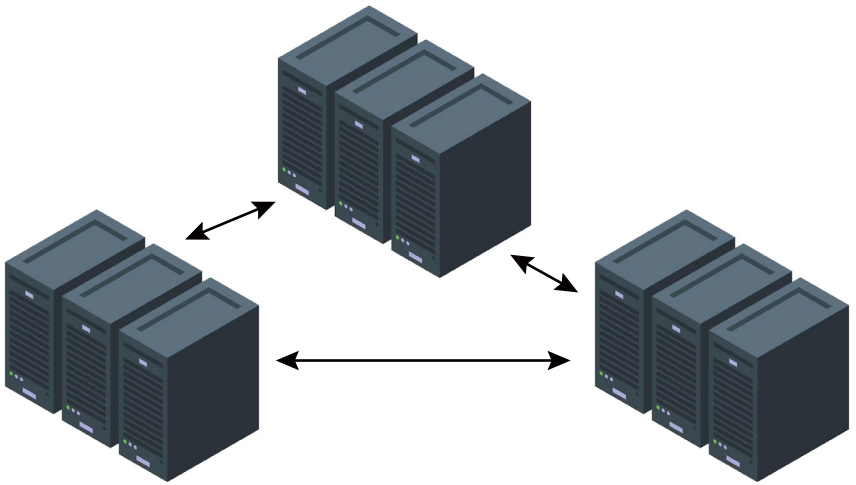
COMP3026E Distributed Systems

1. Overview

James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

Distributed Systems, What?

- A distributed system is a collection of independent computers that appear to the users of the system as a single computer.
  - Multiple computers
  - Connected by a network
  - Doing a common task



Distributed Systems, Why?

- Or, why not one computer to rule them all?
  - Failure
  - Limited resources: CPU, memory, storage, bandwidth …
  - Physical location

Distributed Systems, Where?

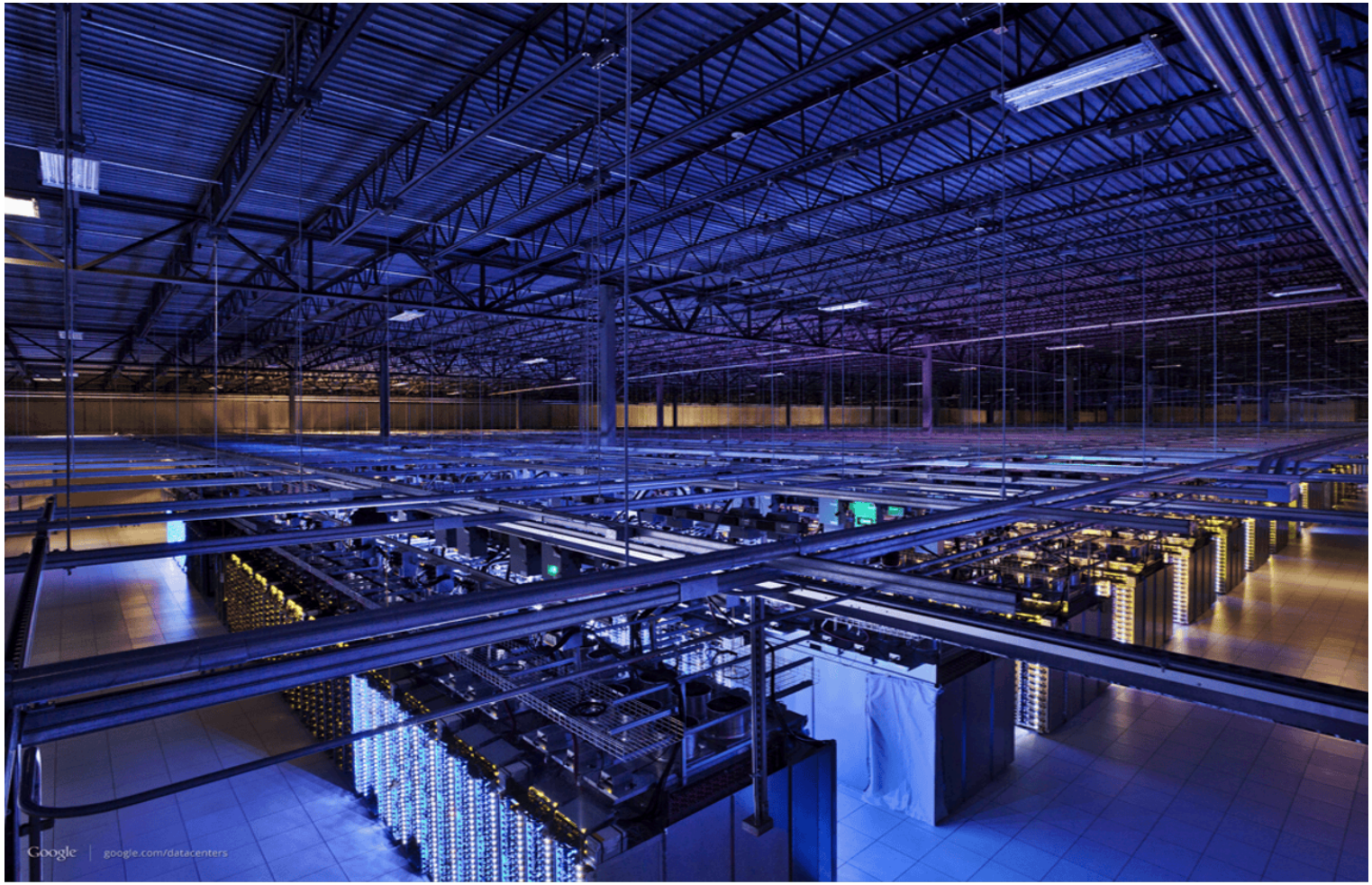
- Web Search (e.g., Google, Bing)
- Shopping (e.g., Amazon, Walmart)
- File Sync (e.g., Dropbox, iCloud)
- Social Networks (e.g., Facebook, Twitter)
- Music (e.g., Spotify, Apple Music)
- Ride Sharing (e.g., Uber, Lyft)
- Video (e.g., Youtube, Netflix)
- Online gaming (e.g., Minecraft, Roblox)
- AI Models (OpenAI, Anthropic)



Backrub (Google) 1997



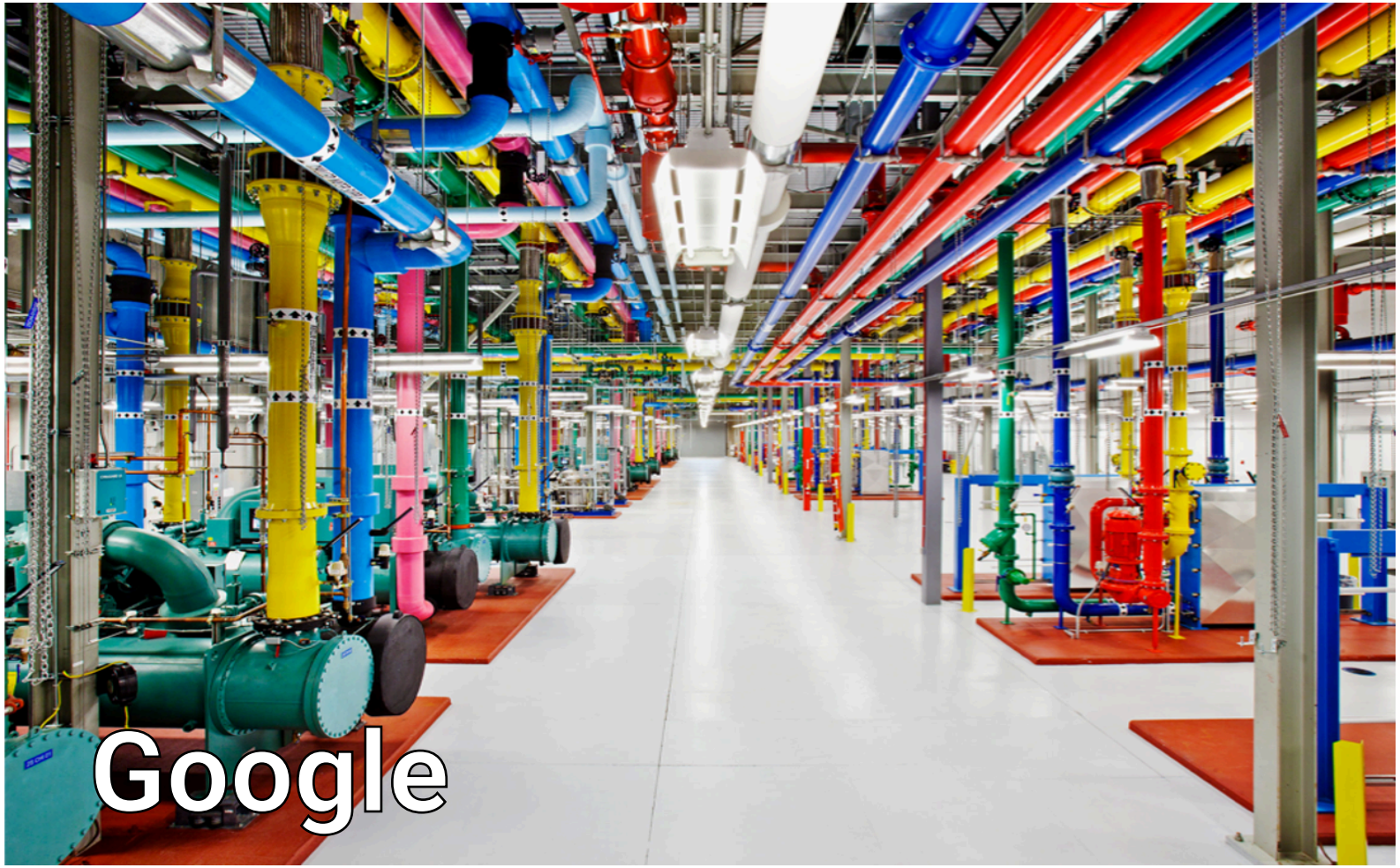
Google 2012



“The Cloud” is NOT amorphous



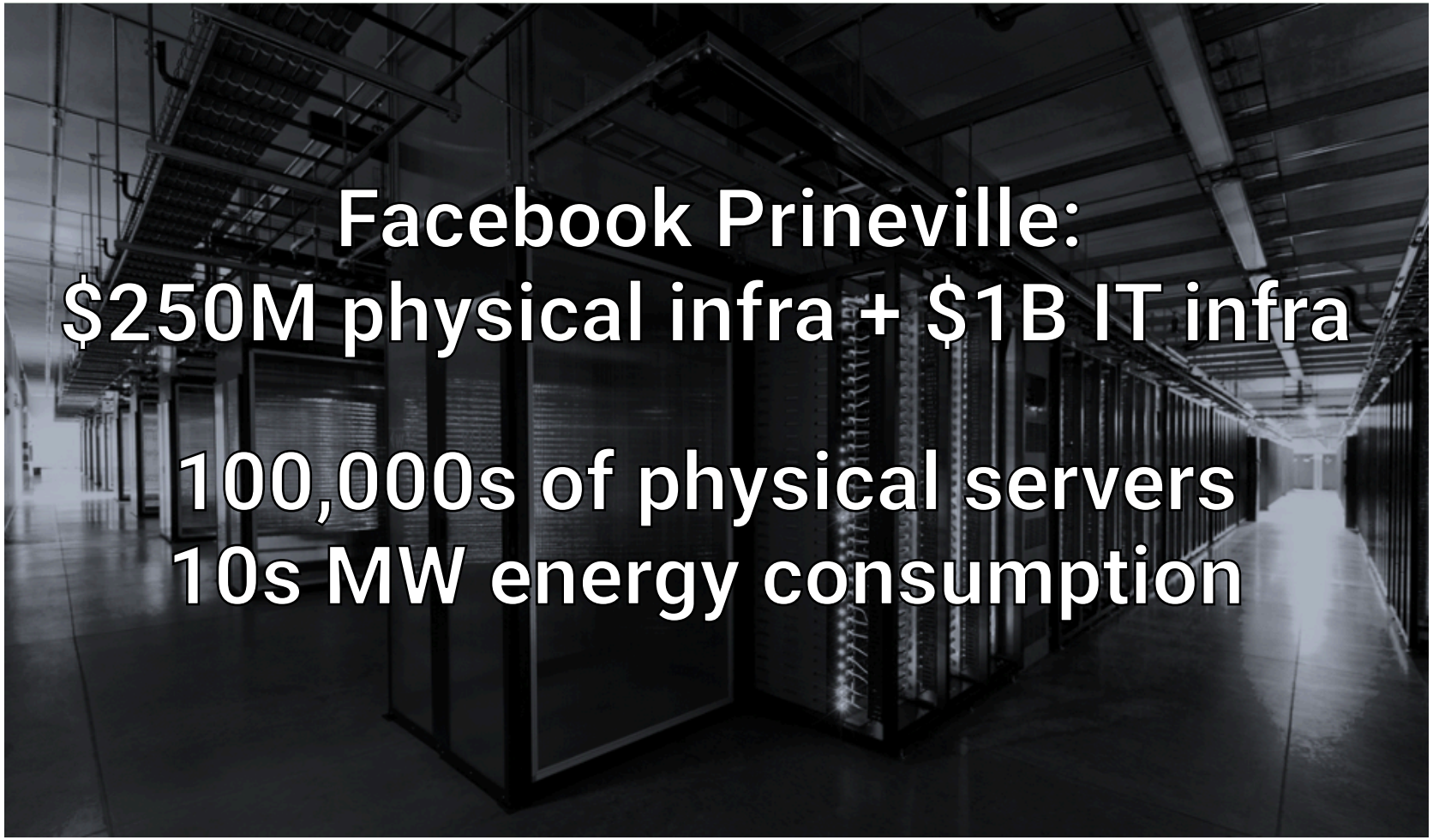
“The Cloud” is NOT Amorphous



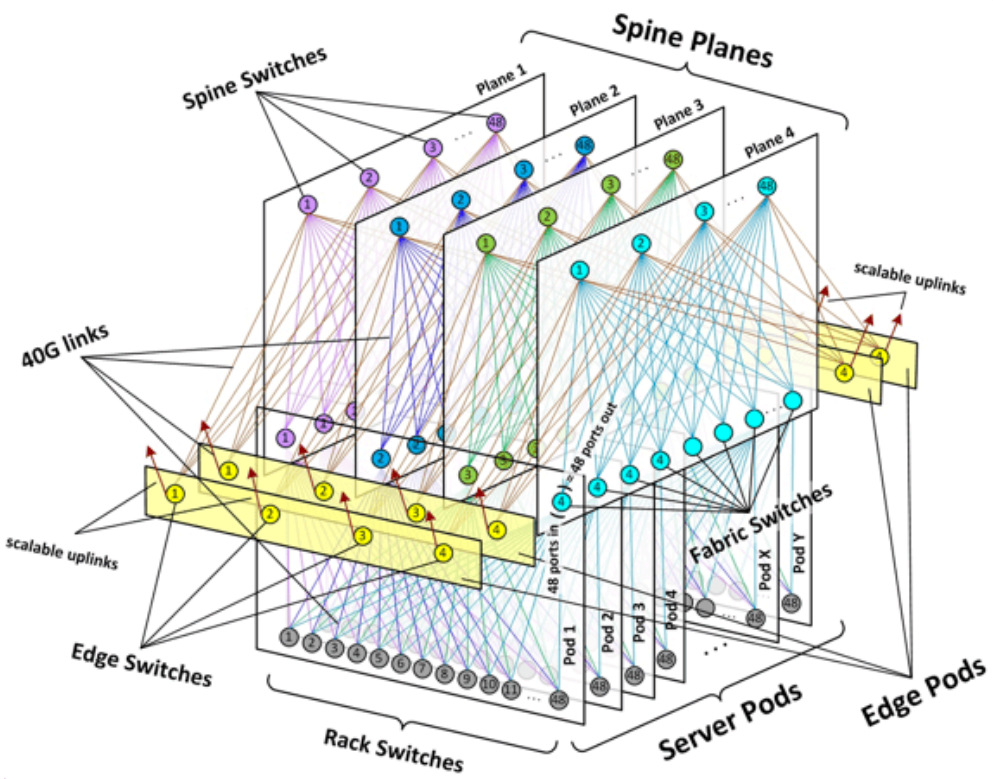
“The Cloud” is NOT Amorphous



Everything Changes At Scale

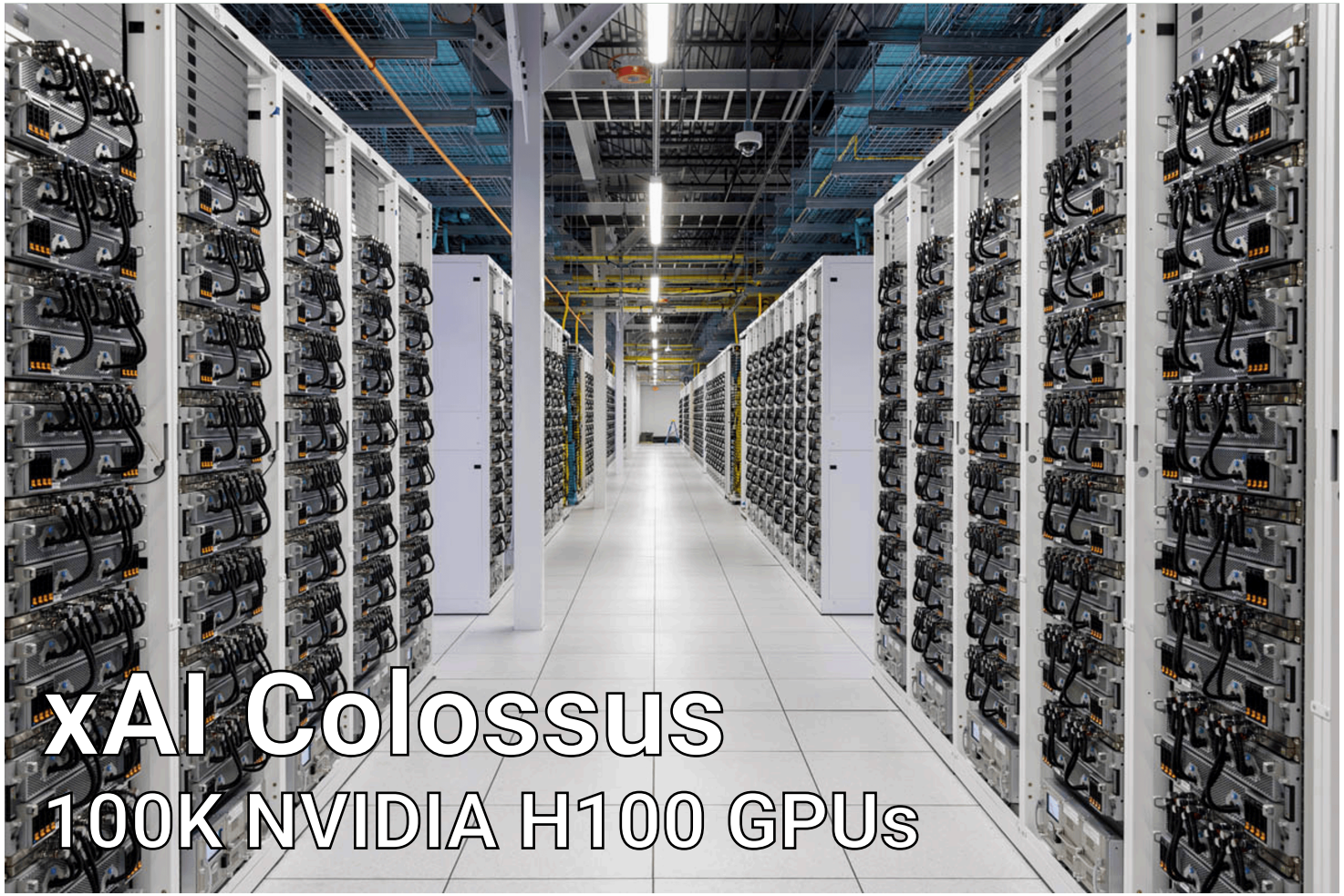


Everything Changes At Scale



- “Pods provide 7.68Tbps to backplane”

Everything Changes At Scale



Distributed Systems Goal

- Service with higher-level abstractions/interface
  - e.g., file system, database, key-value store, programming model, …

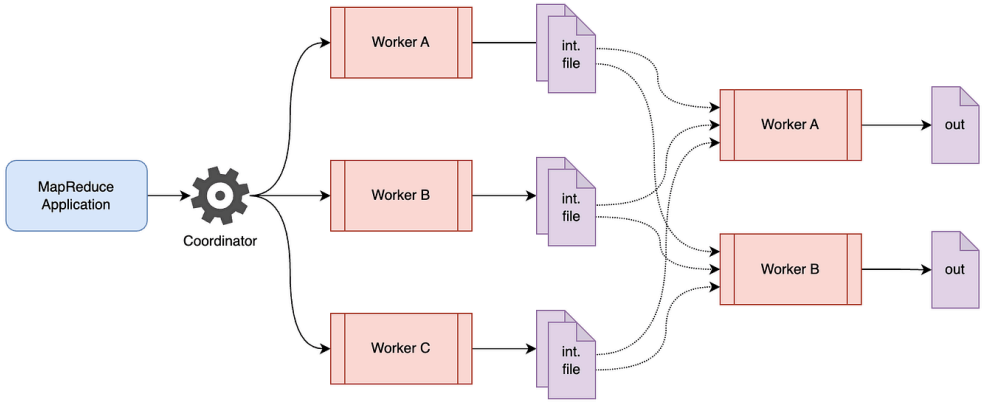
Distributed Systems Goal

- Service with higher-level abstractions/interface
  - e.g., file system, database, key-value store, programming model, …
- Hide complexity
  - Scalable (scale-out)
  - Reliable (fault-tolerant)
  - Well-defined semantics (consistent)

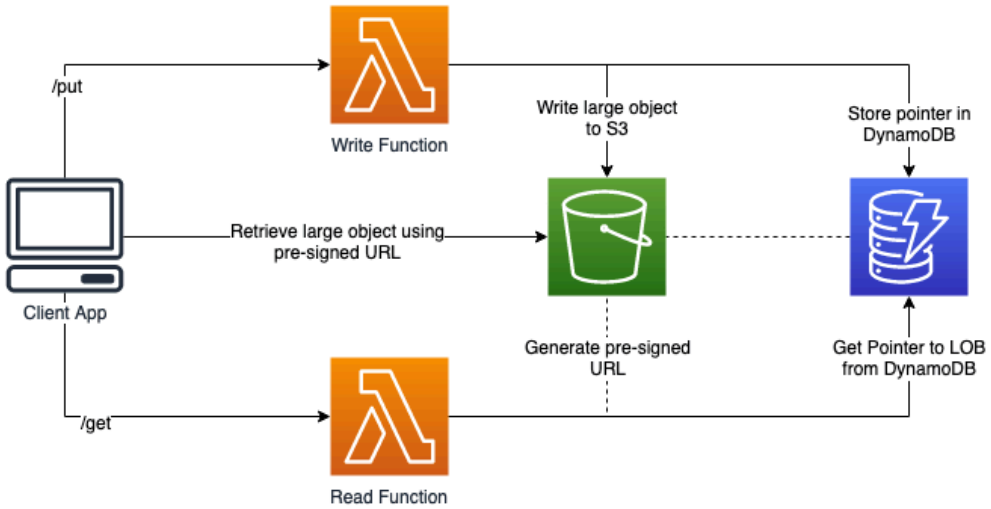
Do “heavy lifting” so app developer doesn’t need to.

Scalable Systems in this Class

- Scale **computation** across many machines
  - MapReduce
- Scale **storage** across many machines
  - Dynamo, COPS, Spanner



MapReduce



Dynamo

Fault Tolerant Systems in this Class

- **Retry** on another machine
  - MapReduce, Distributed Snapshots
- Maintain **replicas** on multiple machines
  - Primary-backup replication
  - Paxos
  - RAFT
  - Bayou
  - Dynamo, COPS, Spanner

Consistency: Abstractions and Guarantees

- **Eventual** Consistency
  - Dynamo

Consistency: Abstractions and Guarantees

- **Eventual** Consistency
  - Dynamo
- **Causal** Consistency
  - Bayou, COPS

Consistency: Abstractions and Guarantees

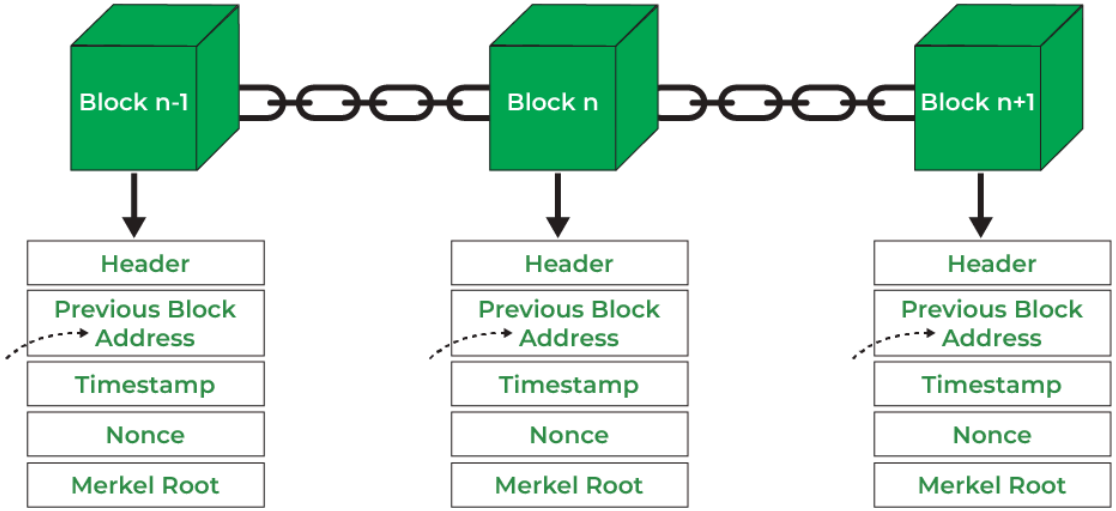
- **Eventual** Consistency
  - Dynamo
- **Causal** Consistency
  - Bayou, COPS
- **Linearizability**
  - Paxos, RAFT, Primary-backup replication

Consistency: Abstractions and Guarantees

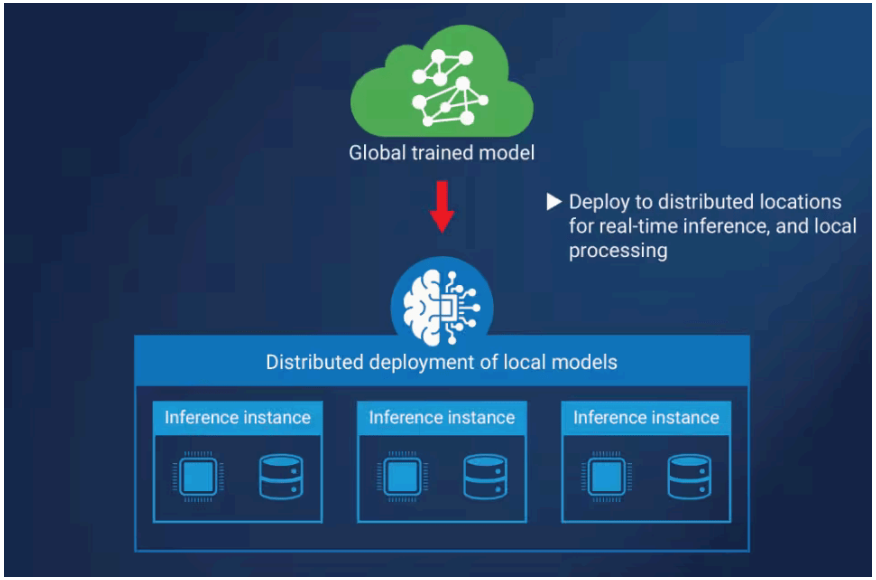
- **Eventual** Consistency
  - Dynamo
- **Causal** Consistency
  - Bayou, COPS
- **Linearizability**
  - Paxos, RAFT, Primary-backup replication
- Strict **Serializability**
  - 2PL, Spanner

Advanced Topics

- **Blockchain** as distributed systems
- **AI inference** in distributed systems



Blockchain



Distributed AI Inference

Learning Objectives

- Reasoning about **concurrency**
- Reasoning about **failure**
- Reasoning about **performance**

Learning Objectives

- Reasoning about **concurrency**
- Reasoning about **failure**
- Reasoning about **performance**

- **Building** systems that correctly handle concurrency and failure

Learning Objectives

- Reasoning about **concurrency**
- Reasoning about **failure**
- Reasoning about **performance**
- **Building** systems that correctly handle concurrency and failure
- Knowing specific system **designs** and design components

Research Results Matter: NoSQL

Consistent Hashing and Random Trees:
Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*

David Karger[1] Eric Lehman[1] Tom Leighton[1,2] Matthew Levine[1] Daniel Lewin[1]
Rina Panigrahy[1]

Abstract

We describe a family of caching protocols for distrib-uted networks that can be used to decrease or eliminate the occurrence of hot spots in the network. Our protocols are particularly designed for use with very large networks such as the Internet, where delays caused by hot spots can be severe, and where it is not feasible for every server to have complete information about the current state of the entire network. The protocols are easy to implement using existing network protocols such as TCP/IP, and require very little overhead. The protocols work with local control, make efficient use of existing resources, and scale gracefully as the network grows.

Our caching protocols are based on a special kind of hashing that we call *consistent hashing*. Roughly speaking, a consistent hash function is one which changes minimally as the range of the function changes. Through the development of good consistent hash functions, we are able to develop caching protocols which do not require users to have a current or even consistent view of the network. We believe that consistent hash functions may eventually prove to be useful in other applications such as distributed name servers and/or quorum systems.

1 Introduction

In this paper, we describe caching protocols for distributed networks that can be used to decrease or eliminate the occurrences of "hot spots". *Hot spots* occur any time a large number of clients it was originally configured to handle. In fact, a site may receive so many requests that it becomes "swamped," which typically renders it unusable. Besides making the one site inaccessible, heavy traffic destined to one location can congest the network near it, interfering with traffic at nearby sites.

As use of the Web has increased, so has the occurrence and impact of hot spots. Recent famous examples of hot spots on the Web include the JPL site after the Shoemaker-Levy 9 comet struck Jupiter, an IBM site during the Deep Blue-Kasparov chess tournament, and several political sites on the night of the election. In some of these cases, users were denied access to a site for hours or even days. Other examples include sites identified as "Web-site-of-the-day" and sites that provide new versions of popular software.

Our work was originally motivated by the problem of hot spots on the World Wide Web. We believe the tools we develop may be relevant to many client-server models, because centralized servers on the Internet such as Domain Name servers, Multicast servers, and Content Label servers are also susceptible to hot spots.

1.1 Past Work

Several approaches to overcoming the hot spots have been proposed. Most use some kind of replication strategy to store copies of hot pages throughout the Internet; this spreads the work of serving a hot page across several servers. In one approach, already in wide use, several clients share a *proxy cache*. All user requests are forwarded through the proxy, which tries to keep copies of frequently

**Managing Update Conflicts in Bayou,
a Weakly Connected Replicated Storage System**

Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers,
Mike J. Spreitzer and Carl H. Hauser

Computer Science Laboratory
Xerox Palo Alto Research Center
Palo Alto, California 94304 U.S.A.

Abstract

Bayou is a replicated, weakly consistent storage system designed for a mobile computing environment that includes portable machines with less than ideal network connectivity. To maximize availability, users can read and write any accessible replica. Bayou's design has focused on supporting application-specific mechanisms to detect and resolve the update conflicts that naturally arise in such a system, ensuring that replicas move towards eventual consistency, and defining a protocol by which the resolution of update conflicts stabilizes. It includes novel methods for conflict detection, called dependency checks, and per-write conflict resolution based on client-provided merge procedures. To guarantee eventual consistency, Bayou servers must be able to roll-back the effects of previously executed writes and redo them according to a global serialization order. Furthermore, Bayou permits clients to observe the results of all writes received by a server, including tentative writes whose conflicts have not been ultimately resolved. This paper presents the motivation for and design of these mechanisms and describes the experiences gained with an initial implementation of the system.

1. Introduction

The Bayou storage system provides an infrastructure for collaborative applications that manages the conflicts introduced by concurrent activity while relying only on the weak connectivity available for mobile computing. The advent of mobile computers, "connectedness" are possible. Groups of computers may be partitioned away from the rest of the system yet remain connected to each other. Supporting disconnected workgroups is a central goal of the Bayou system. By relying only on pair-wise communication in the normal mode of operation, the Bayou design copes with arbitrary network connectivity.

A weak connectivity networking model can be accommodated only with weakly consistent, replicated data. Replication is required since a single storage site may not be reachable from mobile clients or within disconnected workgroups. Weak consistency is desired since any replication scheme providing one copy serializability [6], such as requiring clients to access a quorum of replicas or to acquire exclusive locks on data that they wish to update, yields unacceptably low write availability in partitioned networks [5]. For these reasons, Bayou adopts a model in which clients can read and write to any replica without the need for explicit coordination with other replicas. Every computer eventually receives updates from every other, either directly or indirectly, through a chain of pair-wise interactions.

Unlike many previous systems [12, 27], our goal in designing the Bayou system was *not* to provide transparent replicated data support for existing file system and database applications. We believe that applications must be aware that they may read weakly consistent data and also that their write operations may conflict with those of other users and applications. Moreover, applications must be involved in the detection and resolution of conflicts since these naturally depend on the semantics of the application.

To this end, Bayou provides system support for application-specific conflict detection and resolution. Previous systems, such

Research Results Matter: NoSQL

Consisten
Distributed Caching Protocol

David Karger[1] Eric Lehman[1]

Abstract

We describe a family of caching protocols for distrib
that can be used to decrease or eliminate the occurrenc
in the network. Our protocols are particularly designe
very large networks such as the Internet, where dela
hot spots can be severe, and where it is not feasible fo
to have complete information about the current state
network. The protocols are easy to implement using
work protocols such as TCP/IP, and require very lit
The protocols work with local control, make efficient
ing resources, and scale gracefully as the network gre

Our caching protocols are based on a special kin
that we call *consistent hashing*. Roughly speaking,
hash function is one which changes minimally as the
function changes. Through the development of go
hash functions, we are able to develop caching protoc
not require users to have a current or even consisten
network. We believe that consistent hash functions m
prove to be useful in other applications such as dist
servers and/or quorum systems.

1 Introduction

In this paper, we describe caching protocols for di
works that can be used to decrease or eliminate the
of "hot spots". *Hot spots* occur any time a large num

Dynamo: Amazon's Highly Available Key-value Store

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall and Werner Vogels

Amazon.com

ABSTRACT

Reliability at massive scale is one of the biggest challenges we face at Amazon.com, one of the largest e-commerce operations in the world; even the slightest outage has significant financial consequences and impacts customer trust. The Amazon.com platform, which provides services for many web sites worldwide, is implemented on top of an infrastructure of tens of thousands of servers and network components located in many datacenters around the world. At this scale, small and large components fail continuously and the way persistent state is managed in the face of these failures drives the reliability and scalability of the software systems.

This paper presents the design and implementation of Dynamo, a highly available key-value storage system that some of Amazon's core services use to provide an "always-on" experience. To achieve this level of availability, Dynamo sacrifices consistency under certain failure scenarios. It makes extensive use of object versioning and application-assisted conflict resolution in a manner that provides a novel interface for developers to use.

Categories and Subject Descriptors

D.4.2 [**Operating Systems**]: Storage Management; D.4.5 [**Operating Systems**]: Reliability; D.4.2 [**Operating Systems**]: Performance;

General Terms

Algorithms, Management, Measurement, Performance, Design,

One of the lessons our organization has learned from operating Amazon's platform is that the reliability and scalability of a system is dependent on how its application state is managed. Amazon uses a highly decentralized, loosely coupled, service oriented architecture consisting of hundreds of services. In this environment there is a particular need for storage technologies that are always available. For example, customers should be able to view and add items to their shopping cart even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados. Therefore, the service responsible for managing shopping carts requires that it can always write to and read from its data store, and that its data needs to be available across multiple data centers.

Dealing with failures in an infrastructure comprised of millions of components is our standard mode of operation; there are always a small but significant number of server and network components that are failing at any given time. As such Amazon's software systems need to be constructed in a manner that treats failure handling as the normal case without impacting availability or performance.

To meet the reliability and scaling needs, Amazon has developed a number of storage technologies, of which the Amazon Simple Storage Service (also available outside of Amazon and known as Amazon S3), is probably the best known. This paper presents the design and implementation of Dynamo, another highly available and scalable distributed data store built for Amazon's platform.

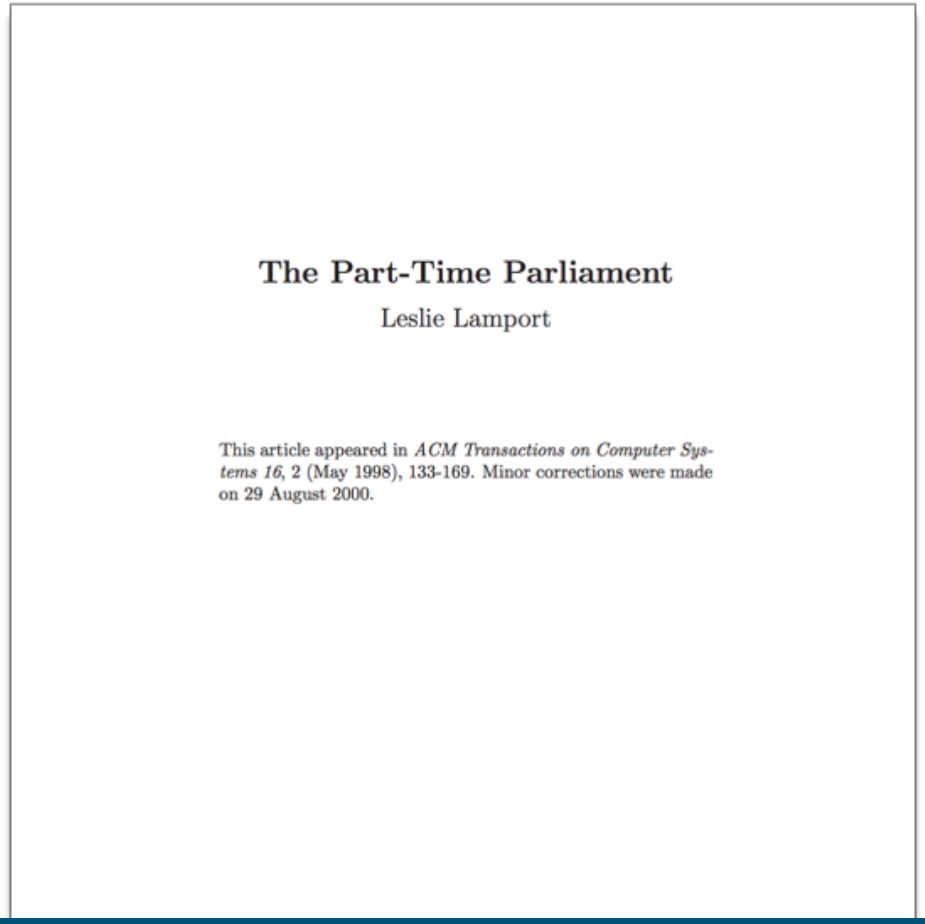
s in Bayou,
Storage System

Petersen, Alan J. Demers,
. Hauser

atory
Center
U.S.A.

lness" are possible. Groups of computers may be parti-
ly from the rest of the system yet remain connected to
. Supporting disconnected workgroups is a central goal
ou system. By relying only on pair-wise communication
mal mode of operation, the Bayou design copes with
etwork connectivity.
k connectivity networking model can be accommodated
weakly consistent, replicated data. Replication is
ince a single storage site may not be reachable from
ents or within disconnected workgroups. Weak consis-
esired since any replication scheme providing one copy
lity [6], such as requiring clients to access a quorum of
to acquire exclusive locks on data that they wish to
elds unacceptably low write availability in partitioned
[5]. For these reasons, Bayou adopts a model in which
read and write to any replica without the need for
ordination with other replicas. Every computer eventu-
es updates from every other, either directly or indirectly,
chain of pair-wise interactions.
many previous systems [12, 27], our goal in designing
system was *not* to provide transparent replicated data
existing file system and database applications. We
applications must be aware that they may read weakly
data and also that their write operations may conflict
of other users and applications. Moreover, applications
volved in the detection and resolution of conflicts since
ally depend on the semantics of the application.
end, Bayou provides system support for application-
nflict detection and resolution. Previous systems, such

Research Results Matter: Paxos



The Part-Time Parliament

Leslie Lamport

This article appeared in *ACM Transactions on Computer Systems 16*, 2 (May 1998), 133-169. Minor corrections were made on 29 August 2000.

Research Results Matter: Paxos



The Part-Time Parliament

Leslie Lamport

This article appeared in *ACM Transactions on Computer Systems 16*, 2 (May 1998), 133-169. Minor corrections were made on 29 August 2000.

Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems

Brian M. Oki
Barbara H. Liskov

Massachusetts Institute of Technology
Laboratory for Computer Science
Cambridge, MA 02139

Abstract

One of the potential benefits of distributed systems is their use in providing highly-available services that are likely to be usable when needed. Availability is achieved through replication. By having more than one copy of information, a service continues to be usable even when some copies are inaccessible, for example, because of a crash of the computer where a copy was stored. This paper presents a new replication algorithm that has desirable performance properties. Our approach is based on the primary copy technique. Computations run at a primary, which notifies its backups of what it has done. If the primary crashes, the backups are reorganized, and one of the backups becomes the new primary. Our method works in a general network with both node crashes and partitions. Replication causes little delay in user computations and little information is lost in a reorganization; we use a special kind of timestamp called a viewstamp to detect lost information.

1 Introduction

One of the potential benefits of distributed systems is their use in providing highly-available services, that is, services that are likely to be up and accessible when needed. Availability is essential to many computer-based services; for example, in airline reservation systems the failure of a single computer can prevent ticket sales for a considerable time, causing a loss of revenue and passenger goodwill.

Availability is achieved through replication. By having more than one copy of important information, the service continues to be usable even when some copies are inaccessible, for example, because of a crash of the computer where a copy was stored. Various replication

Our algorithm runs on a system consisting of nodes connected by a communication network. Nodes are independent computers that communicate with each other only by sending messages over the network. Although both nodes and the network may fail, we assume these failures are not byzantine [24]. Nodes can crash, but we assume they are failstop processors [34]. The network may lose, delay, and duplicate messages, or deliver messages out of order. Link failures may cause the network to partition into subnetworks that are unable to communicate with each other. We assume that nodes eventually recover from crashes and partitions are eventually repaired.

Our replication method assumes a model of computation in which a distributed program consists of *modules*, each of which resides at a single node of the network. Each module contains within it both data objects and code that manipulates the objects; modules can recover from crashes with some of their state intact. No other module can access the data objects of another module directly. Instead, each module provides procedures that can be used to access its objects; modules communicate by means of *remote procedure calls*. Modules that make calls are called *clients*; the called module is a *server*.

Modules are the unit of replication in our method. Ideally, programmers would write programs without concern for availability in some (distributed) programming language that supports our model of computation. The language implementation then uses our technique to replicate individual modules automatically; the resulting programs are highly available.

We assume that computations run as atomic transactions [14]. Our method guarantees the *one-copy serializability* correctness criterion [3, 33]: the concurrent execution of transactions on

Research Results Matter: Paxos

The Chubby lock service for loosely-coupled distributed systems

Mike Burrows, *Google Inc.*

Abstract

We describe our experiences with the Chubby lock service, which is intended to provide coarse-grained locking as well as reliable (though low-volume) storage for a loosely-coupled distributed system. Chubby provides an interface much like a distributed file system with advisory locks, but the design emphasis is on availability and reliability, as opposed to high performance. Many instances of the service have been used for over a year, with several of them each handling a few tens of thousands of clients concurrently. The paper describes the initial design and expected use, compares it with actual use, and explains how the design had to be modified to accommodate the differences.

1 Introduction

This paper describes a *lock service* called Chubby. It is intended for use within a loosely-coupled distributed system consisting of moderately large numbers of small machines connected by a high-speed network. For example, a Chubby instance (also known as a Chubby *cell*) might serve ten thousand 4-processor machines connected by

example, the Google File System [7] uses a Chubby lock to appoint a GFS master server, and Bigtable [3] uses Chubby in several ways: to elect a master, to allow the master to discover the servers it controls, and to permit clients to find the master. In addition, both GFS and Bigtable use Chubby as a well-known and available location to store a small amount of meta-data; in effect they use Chubby as the root of their distributed data structures. Some services use locks to partition work (at a coarse grain) between several servers.

Before Chubby was deployed, most distributed systems at Google used *ad hoc* methods for primary election (when work could be duplicated without harm), or required operator intervention (when correctness was essential). In the former case, Chubby allowed a small saving in computing effort. In the latter case, it achieved a significant improvement in availability in systems that no longer required human intervention on failure.

Readers familiar with distributed computing will recognize the election of a primary among peers as an instance of the *distributed consensus* problem, and realize we require a solution using *asynchronous* communication; this term describes the behaviour of the vast majority of real networks, such as Ethernet or the Internet,

Research Results Matter: MapReduce

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

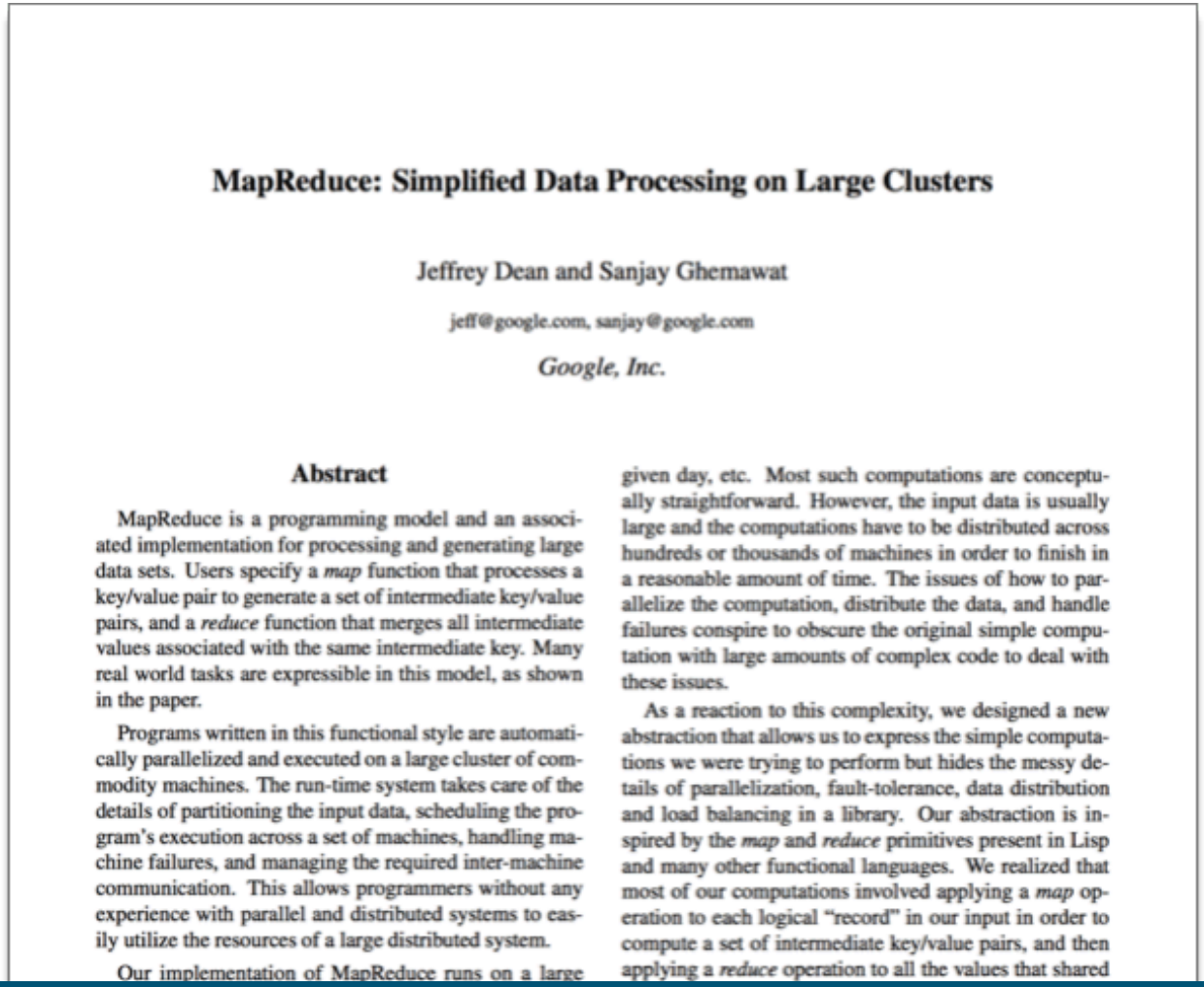
Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the program's execution across a set of machines, handling machine failures, and managing the required inter-machine communication. This allows programmers without any experience with parallel and distributed systems to easily utilize the resources of a large distributed system.

Our implementation of MapReduce runs on a large

given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is inspired by the *map* and *reduce* primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a *map* operation to each logical "record" in our input in order to compute a set of intermediate key/value pairs, and then applying a *reduce* operation to all the values that shared

Research Results Matter: MapReduce



MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the program's execution across a set of machines, handling machine failures, and managing the required inter-machine communication. This allows programmers without any experience with parallel and distributed systems to easily utilize the resources of a large distributed system.

Our implementation of MapReduce runs on a large

given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is inspired by the *map* and *reduce* primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a *map* operation to each logical "record" in our input in order to compute a set of intermediate key/value pairs, and then applying a *reduce* operation to all the values that shared

hadoop

Spark

kafka

Flink

S4 distributed stream computing platform

STORM

Summary of Overview

- Distributed Systems
  - Multiple machines doing something together
  - Pretty much everywhere and everything computing now
- “Systems”
  - Hide complexity and do the heavy lifting (i.e., interesting!)
  - Scalability, fault tolerance, guarantees

Lectures

- Two lectures per week, each 100 minutes
  - 14:00–15:45 on Tuesdays from Week 1 to 8
  - 10:30–12:15 on Thursdays from Week 1 to 8
  - There will be a break in the middle of each lecture
- All lectures will be in this room (H403)

Course Staff and QQ Group

- Lecturer: Prof. James Jianqiao Yu
  - Professor at Harbin Institute of Technology, Shenzhen
  - Email: jqyu@hit.edu.cn
- Teaching Assistant: Mr. Chao Wang
  - Email: ccwang223@qq.com
  - QQ No. 3444813826
- We will use the QQ group for announcements and discussions
  - QQ group No. **1082281381**
  - Please join the QQ group if you haven't already
  - Group password: **3026** (The course number)

Grading

- 20% Lecture Participation
  - Attendance and in-class activities
- 20% Assignments
  - 2 assignments, each 10%
- 60% Final Exam
  - Closed-book, covering all lectures and assignments

Late Assignment Policy

- Late assignment submission policy
  - G: Grade you would have earned if turned in on time
  - 1 day late (including less than a day): $80\% \times G$
  - 2 days late: $60\% \times G$

  ...
  - 5 days late: $0\% \times G$

Copyright

© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.